

Отчет

Предмет: Введение в инженериию данных

Учебная группа: БАСБ253

Группа проекта №: 15

Участники:

Мухаметзянова Дильбар

Шилова Элеонора

Баяндинова Амина

Силин Иван

Содержание

Глава 1. Описание исходных данных.	2
1.1. Дубли в исходных данных.	2
1.2. Отсутствие нескольких курьеров на 1 заказе.	2
1.3 Итог.	3
Глава 2. Инструкция по запуску контейнера в docker и дага для загрузки БД.	3
Глава 3. Обоснование нормализации и выбор нормальной формы.	4
Глава 4. ER диаграмма.	7
Глава 5. DDL скрипты.	8

Глава 1. Описание исходных данных.

1.1. Дубли в исходных данных.

Мы создали таблицу с исходными данными для проверки качества данных. Таблица `delivery_dma_wh.initial_data` (позже таблицу удалили из исходных данных, так как она слишком много весила)

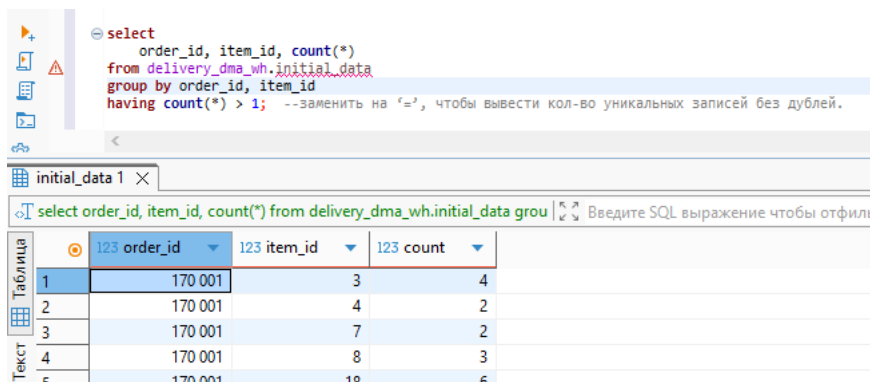
В исходных данных содержится 9 453 279 записей. Из них по ключу `order_id, item_id`:

- 4 093 791 уникальных записей. Из них:
 - 1 506 944 записей без дублей,
 - 2 586 847 уникальных записей с дублями по ключу (всего в исходных данных 7 946 335 записи).

При добавлении `driver_id` в первичный ключ, цифры выше не меняются, следовательно, это поле в ключ не входит.

Исходные данные не очень качественные. В этом случае, явно ошибка на источнике – если это приложение, обрабатывающее заказы, в нем неправильно сохраняются данные. По хорошему, нужно пойти к бизнес-заказчику и разбираться, что не так с источником – править его, чтобы из него приходили уникальные записи. В данном случае заказчика нет, поэтому при загрузке данных будем искусственно отбирать уникальные записи через `distinct/row_number` с сортировкой.

Скрипт для вывода уникальных записей с дублями по ключу:



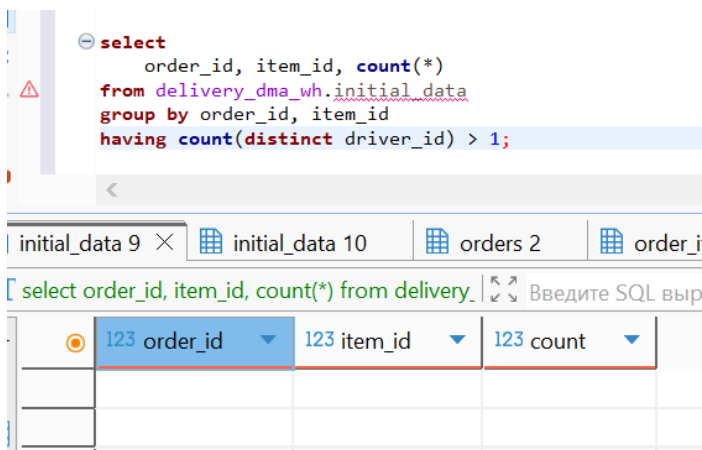
```
select
  order_id, item_id, count(*)
from delivery_dma_wh.initial_data
group by order_id, item_id
having count(*) > 1; --заменить на '=', чтобы вывести кол-во уникальных записей без дублей.
```

	123 order_id	123 item_id	123 count
1	170 001	3	4
2	170 001	4	2
3	170 001	7	2
4	170 001	8	3
5	170 001	18	6

1.2. Отсутствие нескольких курьеров на 1 заказе.

В описании данных написано, что “возможны смены курьера. Тогда для курьера, снятого с доставки, `delivered_at` не проставляется.”.

При проверке есть-ли заказы, в которых указано два курьера, выяснилось, что таких случаев нет:



```
select
  order_id, item_id, count(*)
from delivery_dma_wh.initial_data
group by order_id, item_id
having count(distinct driver_id) > 1;
```

	123 order_id	123 item_id	123 count
--	--------------	-------------	-----------

	<pre>select * from delivery_dma_wh.initial_data where l=1 and user_id = 64538 and to_char(created_at, 'YY-MM-DD') = '25-07-25';</pre>									
	initial_data_6	initial_data_9	initial_data_10	orders_2	order_items_3	initial_data_1	initial_data_4	initial_data_5	initial_data_7	initial_data_8
	select * from delivery_dma_wh.initial_data where l=1 and user_id = 64538 Введите SQL выражение чтобы отфильтровать результаты									
	123 driver_id	123 order_id	123 user_id	AZ user_phone	AZ address_text	created_at	paid_at	delivery_started_at	delivered_at	
1	2 021	3 890 002	64 538	8 332 015 75 84	Орел, апл. 50 лер Октября 14	2025-07-25 06:57:53.202	2025-07-25 10:06:53.202	2025-07-25 07:22:53.202		[NULL]
2	2 021	3 890 002	64 538	8 332 015 75 84	Орел, апл. 50 лер Октября 14	2025-07-25 06:57:53.202	2025-07-25 10:06:53.202	2025-07-25 07:22:53.202		[NULL]
3	2 021	3 890 002	64 538	8 332 015 75 84	Орел, апл. 50 лер Октября 14	2025-07-25 06:57:53.202	2025-07-25 10:06:53.202	2025-07-25 07:22:53.202		[NULL]
4	2 021	3 890 002	64 538	8 332 015 75 84	Орел, апл. 50 лер Октября 14	2025-07-25 06:57:53.202	2025-07-25 10:06:53.202	2025-07-25 07:22:53.202		[NULL]
5	2 021	3 890 002	64 538	8 332 015 75 84	Орел, апл. 50 лер Октября 14	2025-07-25 06:57:53.202	2025-07-25 10:06:53.202	2025-07-25 07:22:53.202		[NULL]
6	2 021	3 890 002	64 538	8 332 015 75 84	Орел, апл. 50 лер Октября 14	2025-07-25 06:57:53.202	2025-07-25 10:06:53.202	2025-07-25 07:22:53.202		[NULL]
7	2 021	3 890 002	64 538	8 332 015 75 84	Орел, апл. 50 лер Октября 14	2025-07-25 06:57:53.202	2025-07-25 10:06:53.202	2025-07-25 07:22:53.202		[NULL]
8	2 021	3 890 002	64 538	8 332 015 75 84	Орел, апл. 50 лер Октября 14	2025-07-25 06:57:53.202	2025-07-25 10:06:53.202	2025-07-25 07:22:53.202		[NULL]
9	2 021	3 890 002	64 538	8 332 015 75 84	Орел, апл. 50 лер Октября 14	2025-07-25 06:57:53.202	2025-07-25 10:06:53.202	2025-07-25 07:22:53.202		[NULL]
10	2 021	3 890 002	64 538	8 332 015 75 84	Орел, апл. 50 лер Октября 14	2025-07-25 06:57:53.202	2025-07-25 10:06:53.202	2025-07-25 07:22:53.202		[NULL]
11	2 021	3 890 002	64 538	8 332 015 75 84	Орел, апл. 50 лер Октября 14	2025-07-25 06:57:53.202	2025-07-25 10:06:53.202	2025-07-25 07:22:53.202		[NULL]
12	2 021	3 890 002	64 538	8 332 015 75 84	Орел, апл. 50 лер Октября 14	2025-07-25 06:57:53.202	2025-07-25 10:06:53.202	2025-07-25 07:22:53.202		[NULL]
13	2 021	3 890 002	64 538	8 332 015 75 84	Орел, апл. 50 лер Октября 14	2025-07-25 06:57:53.202	2025-07-25 10:06:53.202	2025-07-25 07:22:53.202		[NULL]
14	2 021	3 890 002	64 538	8 332 015 75 84	Орел, апл. 50 лер Октября 14	2025-07-25 06:57:53.202	2025-07-25 10:06:53.202	2025-07-25 07:22:53.202		[NULL]
15	2 021	3 890 002	64 538	8 332 015 75 84	Орел, апл. 50 лер Октября 14	2025-07-25 06:57:53.202	2025-07-25 10:06:53.202	2025-07-25 07:22:53.202		[NULL]
16	2 021	3 890 002	64 538	8 332 015 75 84	Орел, апл. 50 лер Октября 14	2025-07-25 06:57:53.202	2025-07-25 10:06:53.202	2025-07-25 07:22:53.202		[NULL]
17	2 021	3 890 002	64 538	8 332 015 75 84	Орел, апл. 50 лер Октября 14	2025-07-25 06:57:53.202	2025-07-25 10:06:53.202	2025-07-25 07:22:53.202		[NULL]
18	2 021	3 890 002	64 538	8 332 015 75 84	Орел, апл. 50 лер Октября 14	2025-07-25 06:57:53.202	2025-07-25 10:06:53.202	2025-07-25 07:22:53.202		[NULL]

Смена курьеров в исходных данных работает криво.

Далее исходные данные проверяться не будут, мы напишем запрос с row number и сортировкой, чтобы отбирать уникальные по ключу записи. Сортировка будет сделана по убыванию количества товара в заказе ("item quantity"), так как в дублях чаще всего отличается именно это поле.

- Необходимо минимум 6 гб памяти у докера. Из них 4гб будет использовано PySpark.
- Инструкция по запуску докера:
 - Необходимо открыть терминал в папке “Folder for Docker launch”
 - Запустить скрипт **docker-compose up --build**
- Далее нужно открыть в браузере страницу с адресом: <http://localhost:5050>. Так можно подключиться к pgAdmin и посмотреть реализацию базы данных.
- Далее нужно открыть в браузере страницу с адресом: <http://localhost:8081>. Так можно подключиться к Airflow. Пользователь: admin, пароль: admin.
 - Здесь отобразится даг нашего проекта с названием delivery_etl_pipeline. Необходимо его запустить – это загрузит данные в слои DDS и DMA.

Глава 3. Обоснование нормализации и выбор нормальной формы.

Исходные данные предоставлены в денормализованном виде (плоская таблица), где каждая строка содержит полную информацию о товаре в рамках конкретного заказа. Из-за такого подхода возникает несколько проблем:

1. **Избыточность данных:** Информация о пользователе (телефон), магазине (адрес), товаре (название, категория) и заказе (даты, скидки) многократно дублируется для каждой товарной позиции в заказе. Это приводит к неэффективному использованию дискового пространства.
2. **Аномалии обновления:** При изменении какой-либо информации (например, номера телефона пользователя или адреса магазина) потребуется обновить множество записей, что увеличивает риск возникновения ошибок и несогласованности данных.
3. **Аномалии вставки и удаления:** Невозможно добавить информацию о новом магазине, если у него еще не было заказов. При удалении единственного заказа из магазина может быть потеряна вся информация о нем.

1.2. Цель нормализации

Целью нормализации является устранение перечисленных проблем путем организации данных в реляционной базе данных. Основные задачи — минимизировать избыточность, обеспечить целостность данных и упростить их дальнейшее обслуживание и использование.

1.3. Выбор нормальной формы: Третья нормальная форма (3НФ)

Для проектирования хранилища данных (DDS-слоя) была выбрана **Третья нормальная форма (3НФ)**. Этот выбор является отраслевым стандартом для построения реляционных хранилищ и обеспечивает оптимальный баланс между устранением избыточности и сложностью запросов.

Для приведения схемы к 3НФ необходимо последовательно удовлетворить требованиям предыдущих форм:

- **Первая нормальная форма (1НФ):** Все атрибуты таблицы должны быть атомарными (неделимыми), а каждая запись должна иметь уникальный первичный ключ. В нашем случае это достигается путем разделения данных на строки и столбцы.
- **Вторая нормальная форма (2НФ):** Таблица должна быть в 1НФ, и все неключевые атрибуты должны полностью зависеть от *всего* первичного ключа (а не от его части, если ключ составной).
- **Третья нормальная форма (3НФ):** Таблица должна быть в 2НФ, и в ней не должно быть транзитивных зависимостей. Это означает, что неключевые атрибуты не должны зависеть от других неключевых атрибутов.

Выбор 3НФ позволяет создать логически понятную, масштабируемую и надежную структуру данных, которая станет прочным фундаментом для построения витрин.

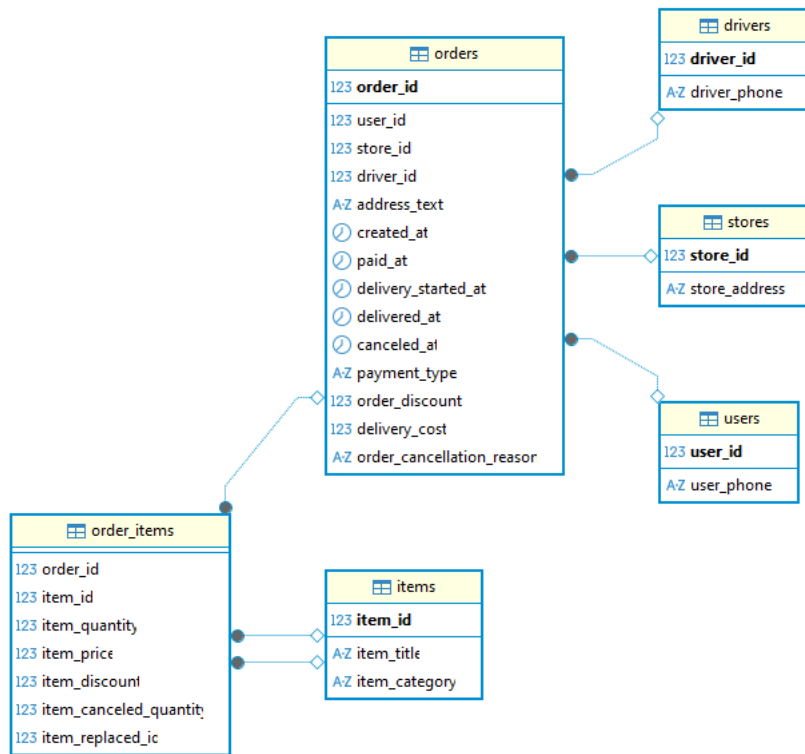
2. Процесс декомпозиции

На основе анализа исходных данных были выделены ключевые логические сущности, которые легли в основу нормализованной схемы. Процесс декомпозиции проходил следующим образом:

1. **Выделение справочников (Dimension Tables):** Были определены независимые сущности, информация о которых дублировалась в исходном файле. Для каждой из них была создана отдельная таблица-справочник:
 - **delivery_dds.users:** Хранит уникальную информацию о пользователях.
 - *Ключ:* **user_id** (Primary Key).
 - *Атрибуты:*
 - **user_phone.** (Атрибут **user_phone** напрямую зависит от **user_id**, удовлетворяя ЗНФ.)
 - **delivery_dds.stores:** Хранит уникальную информацию о магазинах.
 - *Ключ:* **store_id** (Primary Key).
 - *Атрибуты:* **store_address.**
 - **delivery_dds.drivers:** Хранит уникальную информацию о курьерах.
 - *Ключ:* **driver_id** (Primary Key).
 - *Атрибуты:* **driver_phone.**
 - **delivery_dds.items:** Хранит уникальную информацию о товарах.
 - *Ключ:* **item_id** (Primary Key).
 - *Атрибуты:* **item_title, item_category.** Название и категория напрямую зависят от ID товара, транзитивные зависимости отсутствуют.
2. **Создание таблиц фактов (Fact Tables):** После выделения справочников были созданы таблицы, которые хранят "события" (заказы) и связывают сущности между собой.
 - **delivery_dds.orders:** Центральная таблица фактов, хранящая информацию на уровне всего заказа.
 - *Ключ:* **order_id** (Primary Key).
 - *Атрибуты:* Содержит фактические данные о заказе (даты, скидки, стоимость доставки) и внешние ключи (**user_id, store_id, driver_id**), которые ссылаются на соответствующие справочники.
 - **delivery_dds.order_items:** Таблица, детализирующая состав каждого заказа. Она отражает связь "многие-ко-многим" между заказами и товарами.
 - *Ключ:* **order_id, item_id.**
 - *Атрибуты:* Хранит метрики, относящиеся к конкретному товару в конкретном заказе (**item_quantity, item_price, item_discount** и др.). Эти атрибуты полностью зависят от связки "заказ-товар", что удовлетворяет 2НФ и 3НФ.

Такая структура полностью исключает аномалии обновления, вставки и удаления, обеспечивает целостность данных за счет использования внешних ключей и служит надежной основой для дальнейшего анализа.

Глава 4. ER диаграмма.



Глава 5. DDL скрипты.

Для того, чтобы DDL скрипты отображались корректно, вставляем их скриншотами. Сами DDL скрипты лежат в файле init.sql в zip папке для докера.

```
-- DDL скрипты

--Создаем схемы для каждого слоя данных
CREATE SCHEMA IF NOT EXISTS delivery_dds; -- Data Detail Store (DWH)
CREATE SCHEMA IF NOT EXISTS delivery_dma; -- Data Marts

-- Таблица Users
-- Хранится информация о пользователях
CREATE TABLE IF NOT EXISTS delivery_dds.users (
  user_id BIGINT PRIMARY KEY,
  user_phone VARCHAR(50)
);

-- Таблица: stores
-- Хранится информация о магазинах
CREATE TABLE IF NOT EXISTS delivery_dds.stores (
  store_id BIGINT PRIMARY KEY,
  store_address TEXT
);

-- Таблица: drivers
-- Хранится информация о курьерах
CREATE TABLE IF NOT EXISTS delivery_dds.drivers (
  driver_id BIGINT PRIMARY KEY,
  driver_phone VARCHAR(50)
);

-- Таблица: items
-- Хранится информация обо всех товарах
CREATE TABLE IF NOT EXISTS delivery_dds.items (
  item_id BIGINT PRIMARY KEY,
  item_title VARCHAR(255),
  item_category VARCHAR(255)
);

-- Таблица: orders
-- Хранится информация о заказах (центральная таблица)
CREATE TABLE IF NOT EXISTS delivery_dds.orders (
  order_id BIGINT PRIMARY KEY,
  user_id BIGINT REFERENCES delivery_dds.users(user_id),
  store_id BIGINT REFERENCES delivery_dds.stores(store_id),
  driver_id BIGINT REFERENCES delivery_dds.drivers(driver_id),
  address_text TEXT,
  created_at TIMESTAMP,
  paid_at TIMESTAMP,
  delivery_started_at TIMESTAMP,
  delivered_at TIMESTAMP,
  canceled_at TIMESTAMP,
  payment_type VARCHAR(50),
  order_discount DECIMAL(10, 2),
  delivery_cost DECIMAL(10, 2),
  order_cancellation_reason VARCHAR(255)
);

-- Таблица: order_items
-- Хранится информация о связях заказа и товаров
CREATE TABLE IF NOT EXISTS delivery_dds.order_items (
  order_id BIGINT REFERENCES delivery_dds.orders(order_id) ON DELETE CASCADE,
  item_id BIGINT REFERENCES delivery_dds.items(item_id),
  item_quantity INT,
  item_price DECIMAL(10, 2),
  item_discount DECIMAL(10, 2),
  item_canceled_quantity INT,
  item_replaced_id BIGINT REFERENCES delivery_dds.items(item_id)
);

-- Создание индексов для столбцов, которые часто используются для join, чтобы ускорить будущие запросы
CREATE INDEX IF NOT EXISTS idx_orders_user_id ON delivery_dds.orders(user_id);
CREATE INDEX IF NOT EXISTS idx_orders_store_id ON delivery_dds.orders(store_id);
CREATE INDEX IF NOT EXISTS idx_orders_driver_id ON delivery_dds.orders(driver_id);
CREATE INDEX IF NOT EXISTS idx_orders_created_at ON delivery_dds.orders(created_at);
CREATE INDEX IF NOT EXISTS idx_order_items_order_id ON delivery_dds.order_items(order_id);
CREATE INDEX IF NOT EXISTS idx_order_items_item_id ON delivery_dds.order_items(item_id);
```



```

-- =====
-- DDL для витрин данных (Data Marts)
-- =====

-- 1. Витрина заказов (orders_report)
CREATE TABLE IF NOT EXISTS delivery_dma.orders_report (
    id BIGSERIAL PRIMARY KEY,
    -- Разрезы (Dimensions)
    report_date DATE NOT NULL,
    store_id BIGINT NOT NULL,
    store_city VARCHAR(255),
    -- Метрики (Metrics)
    revenue_sum DECIMAL(18, 2),
    turnover_sum DECIMAL(18, 2),
    profit_sum DECIMAL(18, 2),
    created_orders_count INT,
    delivered_orders_count INT,
    canceled_orders_count INT,
    cancellations_after_delivery_count INT,
    service_error_cancellations_count INT,
    unique_buyers_count INT,
    avg_check DECIMAL(18, 2),
    orders_per_buyer DECIMAL(18, 2),
    revenue_per_buyer DECIMAL(18, 2),
    driver_changes_count INT,
    active_drivers_count INT,
    -- Уникальный ключ для идемпотентности
    UNIQUE(report_date, store_id)
);

-- 2. Витрина товаров (items_report)
CREATE TABLE IF NOT EXISTS delivery_dma.items_report (
    id BIGSERIAL PRIMARY KEY,
    -- Разрезы (Dimensions)
    report_date DATE NOT NULL,
    store_id BIGINT NOT NULL,
    store_city VARCHAR(255),
    item_id BIGINT NOT NULL,
    item_category VARCHAR(255),
    -- Метрики (Metrics)
    item_turnover_sum DECIMAL(18, 2),
    ordered_items_quantity INT,
    canceled_items_quantity INT,
    orders_with_item_count INT,
    orders_with_item_cancellation_count INT,
    -- Уникальный ключ для идемпотентности
    UNIQUE(report_date, store_id, item_id)
);

```